
Criterio 3: Documentación y crítica por archivo de código

Documentación técnica de auditoría · Versión LITE (versión resumida) · Un documento por criterio de la rúbrica

Consultor Estadístico Senior · Auditoría de Proyectos de Datos

Universidad Santo Tomás · Ustadistica · 2026-I

Repositorio auditado: `Observatorio_Ministerio_de_Ciencias_Grupo8`

Commit auditado: 1528939

Versión LITE (versión resumida) · 15 de septiembre de 2026

Índice

1. Contexto y guía de lectura	2
1.1. Qué es este documento	2
1.2. Glosario (solo los términos que usa este documento)	2
2. Documentación y crítica por archivo de código	3
2.0.1. src/ingesta/_init_.py	3
2.0.2. src/ingesta/minciencias.py	4
2.0.3. src/ingesta/produccion.py	5
2.0.4. src/Transformacion.py	6
2.0.5. src/modelo/dimensional.py	7
2.0.6. src/analisis/calidad.py	8
2.0.7. src/analisis/diversidad.py	8
2.0.8. src/analisis/genero.py	9
2.0.9. src/analisis/longitudinal.py	10
2.0.10. src/analisis/produccion.py	10
2.0.11. src/analisis/redes.py	11
2.0.12. src/analisis/territorial.py	13
2.0.13. scripts/sprint2_matrices_transicion.py	13
2.0.14. scripts/sprint2_panel_longitudinal.py	14
2.0.15. scripts/sprint5_produccion.py	15
2.0.16. scripts/sprint6_tabla_maestra_ies.py	16
2.0.17. scripts/sprint6_validacion_calidad.py	17
2.0.18. streamlit_app.py	18
3. Relación con los demás criterios	19
4. Conclusiones y veredicto del criterio (versión LITE)	19
4.1. Hallazgos de mayor impacto	19

1 Contexto y guía de lectura

1.1 Qué es este documento

Este documento desarrolla el **criterio 3 de la rúbrica**: *Documentación y crítica por archivo de código*. El repositorio auditado es el Observatorio de investigadores reconocidos del Ministerio de Ciencia, Tecnología e Innovación de Colombia (MinCiencias), que consolida y analiza **6 convocatorias (2013–2021)** del padrón oficial de investigadores y su producción científica. Esta serie cubre los **6 criterios** de la rúbrica en documentos separados, más el documento maestro que los integra.

En resumen: Este criterio evalúa la documentación de los archivos de código: qué hace cada uno, cómo está construido, si está bien hecho y si al ejecutarse produce lo que se buscaba. **Conclusión:** el código está bien construido en general (modular y documentado); 10 de los 16 scripts principales se ejecutan correctamente en un clon del repositorio y los 6 restantes fallan por causas identificadas (datos no versionados y una dependencia no declarada).

Nota metodológica

Versión LITE: este documento resume el criterio (veredicto y hallazgos de mayor impacto). El desarrollo completo está en la versión completa del criterio y en el documento maestro.

Nota metodológica

Cómo leer este documento: cada PDF de esta serie desarrolla *un* criterio de la rúbrica. Los demás criterios se tratan a fondo en sus propios documentos y en el **documento maestro** (`maestro_auditoria.pdf`). Si este es tu primer acercamiento, lee primero la portada y este contexto; al final encontrarás las conclusiones del criterio.

1.2 Glosario (solo los términos que usa este documento)

Esta tabla incluye únicamente los términos técnicos que aparecen en este documento, explicados en lenguaje sencillo:

Tabla 1. Glosario de términos presentes en este documento

Término	Qué significa
HHI (Herfindahl-Hirschman)	Índice que mide si algo está concentrado o repartido. Va de 0 (todo repartido) a 1 (todo en un solo lugar). En el informe se usa para ver si los investigadores se concentran en pocos departamentos.
Matriz de transición	Tabla que muestra con qué probabilidad un investigador pasa de una categoría (p. ej. Junior) a otra (p. ej. Asociado) entre dos convocatorias. Permite ver si el sistema 'sube', 'mantiene' o 'pierde' investigadores.

Término	Qué significa
Tasa de retención	Porcentaje de investigadores que siguen presentes en la siguiente convocatoria. Una retención baja puede indicar abandono o cambio de reglas.
Representación relativa	Cuántas veces aparece un grupo (p. ej. afrocolombianos) en el padrón frente a lo que correspondería según su peso en la población colombiana (DANE). Un valor de 0.3 indica que aparece 3 veces menos de lo esperado.
Modelo dimensional (esquema estrella)	Forma de organizar los datos en tablas de 'hechos' (métricas: quién, cuándo, qué categoría) y 'dimensiones' (descripciones: área, región, institución), para hacer consultas rápidas y reportes.
DuckDB	Base de datos analítica ligera que se ejecuta dentro del propio proyecto (sin instalar un servidor). Se usa para almacenar el modelo dimensional.
Mojibake	Texto ilegible por un problema de codificación (p. ej. 'FÁsica' en lugar de 'Física'). Detectado en el CSV del repositorio.
ETL	Sigla de Extraer, Transformar y Cargar: procesos que extraen los datos de la fuente, los limpian y los dejan listos para analizar.
Markovianidad	Propiedad de un proceso en el que la probabilidad de pasar al siguiente estado solo depende del estado actual (y no de toda la historia). Se usa para validar si las matrices de transición de categorías son adecuadas.
Modelos de supervivencia (Kaplan-Meier / Cox)	Técnicas estadísticas para analizar 'tiempo hasta que ocurre un evento' (p. ej. cuánto tarda un investigador en salir del sistema), incluyendo datos incompletos.
CI/CD y Docker	Automatización de pruebas y despliegue (CI/CD) y empaquetado de la aplicación en un contenedor (Docker) para que funcione igual en cualquier máquina.
Sprint	Ciclo corto de trabajo (en este proyecto, de 2 semanas) con objetivos concretos.

2 Documentación y crítica por archivo de código

Cobertura: 18 de 51 archivos de código documentados en esta versión.

Nota metodológica

Versión LITE: esta versión documenta los archivos más representativos del proyecto en formato compacto. La documentación completa de los 51 archivos (descripción, lógica, fortalezas, debilidades, mejoras, ejecutabilidad) está en el documento maestro y en el PDF del criterio 3 completo.

2.0.1 src/ingesta/__init__.py

Rol: ingest · **Líneas:** 53

Qué hace: Punto de entrada de carga de datos del paquete ingesta. Expone cargar_consolidado() y cargar_produccion(), con prioridad de rutas (CSV crudo Socrata -> XLSX local para investigadores; CSV crudo obligatorio para producción), normaliza nombres de columnas a MAYÚSCULAS y reporta conteos cargados.

Valoración: mejorar — diseño sólido de fallback, pero requiere validación de tipos/duplicados y dependencias declaradas (openpyxl/pyyaml) para que la instalación reproducible funcione.

Hallazgos relevantes:

- src/ingesta/__init__.py:44-48: cargar_produccion() lanza FileNotFoundError sin alternativa; en el clon (sin datos/raw) todos los scripts sprint5/sprint6 de producción mueren en el arranque.

Evidencia en src/ingesta/__init__.py (líneas 44-48):

```

1  if not _PROD.exists():
2      raise FileNotFoundError(
3          "Dataset de producción no encontrado. "
4          "Ejecute: python -m src.ingesta.produccion"
5      )

```

- src/ingesta/__init__.py:26: read_csv(low_memory=False) + read_excel sin dtype explícito -> ID_PERSONA_PR puede inferirse como int en un lado y str en otro, rompiendo el cruce id_persona_pd <-> id_persona_pr.

Evidencia en src/ingesta/__init__.py (línea 26):

```

1  df = pd.read_csv(_RAW, low_memory=False)

```

- src/ingesta/__init__.py:18-36: no verifica duplicados por (ID_PERSONA_PR, ID_CONVOCATORIA) pese a que el README define esa granularidad para el dataset.

Evidencia en src/ingesta/__init__.py (líneas 18-36):

```

1  def cargar_consolidado() -> pd.DataFrame:
2      """Carga el dataset consolidado de investigadores (77 237
3          registros, 30 cols).
4
5          Prioridad: datos/raw/investigadores_consolidado.csv (Socrata)
6                  datos/tarea_join/investigadores_consolidado.xlsx (
7                      Excel local)
8
9          """
10     [...]

```

2.0.2 src/ingesta/minciencias.py

Rol: ingesta · **Líneas:** 70

Qué hace: Descarga el dataset consolidado de investigadores reconocidos de MinCien-
cias desde datos.gov.co (API Socrata, sodapy; recurso bqtm-4y2h) y lo guarda en da-

tos/raw/investigadores_consolidado.csv. Actualiza el campo ultimo_pull del catálogo YAML. CLI con `-limit` (muestra de pruebas) y `-token` (app token Socrata).

Valoración: mantener — simple y funcional; mejorar la validación de codificación y el manejo del catálogo.

Hallazgos relevantes:

- `yaml.dump` reescribe el catálogo y puede alterar su estructura/orden (riesgo de formato).

2.0.3 src/ingesta/produccion.py

Rol: ingesta · **Líneas:** 106

Qué hace: Descarga paginada del dataset de Producción de Grupos (Socrata 33dq-ab5a, 3.166.629 filas) a `datos/raw/produccion_grupos.csv`, con modo `-limit` para pruebas y actualización de metadatos en `datos/catalogo.yaml`. Es la única ingesta del proyecto con paginación real.

Valoración: mantener/mejorar — única ingesta con paginación correcta del proyecto; corregir la dependencia PyYAML declarada y el sobrescrito en modo `limit` para producción segura.

Hallazgos relevantes:

- `src/ingesta/produccion.py:24: import yaml` no cubierto por `pyproject.toml` -> falla en instalación reproducible (probablemente ejecutado ad-hoc con PyYAML instalado manualmente).

Evidencia en src/ingesta/produccion.py (línea 24):

```
1 import yaml
```

- `src/ingesta/produccion.py:47-53: en modo limit` `main()` igualmente guarda el `DataFrame` como CSV 'completo' en SALIDA: si alguien corre `-limit` sin querer, sobrescribe el archivo crudo con una muestra.

Evidencia en src/ingesta/produccion.py (líneas 47-53):

```
1 if limit:
2     registros = client.get(SOCRATA_ID, limit=limit)
3     client.close()
4     df = pd.DataFrame.from_records(registros)
5     log.info("Descargados (modo limit=%d): %d filas, %d
6             columnas",
7             limit, len(df), len(df.columns))
[...]
```

- `src/ingesta/produccion.py:59-65: si la última página devuelve exactamente PAGE_SIZE` y la siguiente viene vacía, el loop hace una petición extra vacía (ineficiencia menor, no error).

Evidencia en src/ingesta/produccion.py (líneas 59-65):

```

1         chunk = client.get(SOCRATA_ID, limit=PAGE_SIZE, offset=
           offset, order=":id")
2         if not chunk:
3             break
4         paginas.append(pd.DataFrame.from_records(chunk))
5         offset += len(chunk)
6         if len(chunk) < PAGE_SIZE:
7     [...]
```

2.0.4 src/Transformacion.py

Rol: transformacion · **Líneas:** 222

Qué hace: Núcleo de limpieza y normalización del dataset consolidado de investigadores (77.237 registros). Aplica limpieza de texto (strip + mayúsculas), parseo robusto del año de convocatoria (fecha, serial de Excel o regex de 4 dígitos), estandarización de género y tratamiento de faltantes (mediana para numéricas, 'NO REPORTADO' para categóricas). Entrada: DataFrame crudo de ingesta.cargar_consolidado(); salida: DataFrame transformado listo para análisis.

Valoración: mejorar — núcleo correcto y bien estructurado; debe mejorarse la trazabilidad de imputaciones y la corrección del año nominal 2018 antes de seguir usándolo en un informe de calidad de datos.

Hallazgos relevantes:

- Transformacion.py:107-109: la regex `r'\{\}b(20[0-9]{2})\{\}b'` solo captura años 2000-2099; correcto para el dominio, pero un año fuera de rango quedaría como NaT sin aviso.

Evidencia en Transformacion.py (líneas 107-109):

```

1         # Intento 3: año de 4 dígitos embebido en el texto
2         anio_regex = s.str.extract(r"\b(20[0-9]{2})\b")[0]
3         fecha_regex = pd.to_datetime(anio_regex, format="%Y", errors="
           coerce")
```

- Transformacion.py:99-100: `pd.to_datetime(s, dayfirst=True)` sobre un texto numérico tipo '640' (id de convocatoria) produciría NaT; depende del contenido real de ANO_CONVO, que el catálogo documenta como año o timestamp.

Evidencia en Transformacion.py (líneas 99-100):

```

1         fecha = pd.to_datetime(s, dayfirst=True, errors="coerce")
```

- Transformacion.py:140-141: si una columna numérica fuera 100 % NaN, `mediana=NaN` y `fillna(NaN)` no corrige nada (caso límite no manejado).

Evidencia en Transformacion.py (líneas 140-141):

```

1         mediana = df[col].median()
2         df[col] = df[col].fillna(mediana)
```

2.0.5 src/modelo/dimensional.py

Rol: modelado · **Líneas:** 286

Qué hace: Construye el modelo dimensional (Issue #14) en DuckDB: 7 dimensiones (convocatoria, investigador, categoría, área, territorio, formación, institución) + fact_clasificacion (investigador x convocatoria) + tabla puente N:N bridge_hecho_institucion. Entrada: DataFrame transformado; salida: datos/processed/observatorio.duckdb (gitignored).

Valoración: mejorar — modelo útil y correcto en granularidad, pero necesita SK/constraints para ser un 'modelo estrella' real y corregir el snapshot cronológico del investigador antes de citarlo como modelo dimensional canónico.

Hallazgos relevantes:

- src/modelo/dimensional.py:69-74: sort solo por ID_PERSONA_PR -> 'keep first' no es la primera aparición temporal; los atributos del snapshot (género, región) pueden pertenecer a una convocatoria posterior.

Evidencia en src/modelo/dimensional.py (líneas 69-74):

```

1      dim = (
2          df[cols]
3          .sort_values("ID_PERSONA_PR")
4          .drop_duplicates(subset=["ID_PERSONA_PR"], keep="first")
5          .reset_index(drop=True)
6      )

```

- src/modelo/dimensional.py:41-44: _drop_and_create nunca se usa (código muerto).

Evidencia en src/modelo/dimensional.py (líneas 41-44):

```

1  def _drop_and_create(con: duckdb.DuckDBPyConnection, ddl: str) ->
    None:
2      table = ddl.split("(")[0].split()[-1]
3      con.execute(f"DROP TABLE IF EXISTS {table}")
4      con.execute(ddl)

```

- src/modelo/dimensional.py:192-197: el merge por columnas geográficas con NaN (código DANE ausente) no empareja en pandas (NaN != NaN) -> fact con sk_territorio NULL para filas sin geografía, sin contarlas ni advertirlo.

Evidencia en src/modelo/dimensional.py (líneas 192-197):

```

1      fact = fact.merge(
2          dim_territorio[["sk_territorio", *geo_cols]],
3          on=geo_cols,
4          how="left",
5      )
6      fact = fact.drop(columns=geo_cols)

```

- `src/modelo/dimensional.py:52-56`: construir `dim_convocatoria` conserva `ANO_CONVOCATORIA` = 2019 para la convocatoria id 20 (año nominal 2018), propagando la anomalía de datos documentada al modelo.

Evidencia en `src/modelo/dimensional.py` (líneas 52-56):

```

1     cols = ["ID_CONVOCATORIA", "NME_CONVOCATORIA", "ANO_CONVOCATORIA",
2           , "ANO_CONVOCATORIA_FECHA"]
3     dim = df[cols].drop_duplicates(subset=["ID_CONVOCATORIA"]).
4         copy()
5     dim = dim.sort_values("ANO_CONVOCATORIA").reset_index(drop=True)
6     dim.columns = dim.columns.str.lower()
7     return dim

```

2.0.6 `src/analisis/calidad.py`

Rol: analisis · **Líneas:** 105

Qué hace: Validación documentada de la calidad de los datos del padrón, con foco en atípicos de edad (`EDAD_ANOS_PR`). Documenta explícitamente la decisión metodológica (eliminación frente a imputación por mediana del grupo) para que cualquier auditor pueda reproducirla y discutirla. Expone `detectar_atipicos_edad`, `resumen_tratamiento`, `filtrar` (método elegido) e `imputar_mediana` (alternativa comparativa).

Valoración: mejorar — lógica correcta y bien documentada; corregir la inconsistencia documental y ampliar la instrumentación de calidad.

Hallazgos relevantes:

- Docstring (línea ~10): 'diez registros' vs 22 reales — inconsistencia que puede confundir al lector del informe.

2.0.7 `src/analisis/diversidad.py`

Rol: analisis · **Líneas:** 162

Qué hace: Análisis de variables de diversidad (Issue #20): cobertura temporal de `ID_VICTIMA_CONFLICTO`, `TXT_GRUPO_ETNICO` y `TXT_POBLACION_DISCA`, distribución por categoría, comparación de subrepresentación frente a DANE 2018/RUV 2021, cruce interseccional género x etnia y categorías por minoría. Entrada: DataFrame transformado (mayúsculas aplicadas por `src/Transformacion.py`); salida: DataFrames exportados a `evidencias/diversidad_*.csv`.

Valoración: mejorar — hallazgos valiosos y bien enmarcados; prioridad alta: corregir el denominador de `comparar_dane()`, alinear valores nulos con `Transformacion.py` y robustecer comparaciones de cadenas.

Hallazgos relevantes:

- `src/analisis/diversidad.py:78-79`: contradicción comentario-código sobre 'NINGUN GRUPO ETNICO' en el denominador de `comparar_dane()` — diluye la subrepresentación de minorías (verificado por lectura directa).

Evidencia en `src/analisis/diversidad.py` (líneas 78-79):

```
1      # Etnia: excluir "NINGUN GRUPO ETNICO" del denominador para
      ver minorias
2      etnia = df_2021[df_2021["TXT_GRUPO_ETNICO"] != "NO DISPONIBLE"
      ]
```

- `src/analisis/diversidad.py:145`: `d['FEMENINO']` puede lanzar `KeyError` solo si NINGÚN grupo étnico tiene mujeres (con `unstack(fill_value=0)` la columna no se crea en ese caso) — crash en caso límite (reproducido).

Evidencia en `src/analisis/diversidad.py` (línea 145):

```
1      .assign(pct_femenino=lambda d: (d["FEMENINO"] / d["total"]
      * 100).round(1))
```

- `src/analisis/diversidad.py:44-46`: los NaN convertidos a 'NO REPORTADO' por `Transformacion.py` no se cuentan como nulos -> `pct_no_registra` subestimado y `cobertura` sobreestimada para registros con faltante real.

Evidencia en `src/analisis/diversidad.py` (líneas 44-46):

```
1      pct_nulo = (grupo[col] == val_nulo).mean() * 100
2      fila[f"{col}_pct_no_registra"] = round(pct_nulo, 2)
3      fila[f"{col}_pct_cobertura"] = round(100 - pct_nulo,
      2)
```

- `src/analisis/diversidad.py:121`: comparación 'SÍ' sensible a acentos; no es un bug activo (los datos traen 'Sí' y `transformar()` lo pone en mayúsculas), pero fallaría si la fuente usara 'Si' sin tilde.

Evidencia en `src/analisis/diversidad.py` (línea 121):

```
1      n_vict = (vict["ID_VICTIMA_CONFLICTO"] == "SÍ").sum()
```

2.0.8 `src/analisis/genero.py`

Rol: análisis · **Líneas:** 85

Qué hace: Análisis de género por área OCDE (Issue #22): porcentaje femenino, tablas pivote y brecha de género por convocatoria a 3 niveles de área. Entrada: DataFrame transformado con `NME_GENERO_PR` estandarizado y `ANO_CONVO_INT`; salida: DataFrames tidy/pivot listos para exportar a `evidencias/genero_*.csv` y graficar.

Valoración: mantener — análisis correcto y bien documentado; mejoras menores de robustez (grupos vacíos) y limpieza de redundancia.

Hallazgos relevantes:

- `src/analisis/genero.py:32-33`: `conteos.get('FEMENINO', 0)` funciona porque `DataFrame.get` acepta default escalar, pero si un grupo queda sin filas reportadas, `n_total=0` y `pct_femenino=NaN` (división por cero) sin advertencia — caso límite real si una convocatoria tuviera 100 % NO REPORTADO.

Evidencia en `src/analisis/genero.py` (líneas 32-33):

```
1 resultado["n_femenino"] = conteos.get("FEMENINO", 0)
2 resultado["n_masculino"] = conteos.get("MASCULINO", 0)
```

- `src/analisis/genero.py:67`: `pct_masculino = 1 - pct_femenino` asume universo binario; correcto tras el filtro, pero la columna `n_masculino` ya existe y no se usa (inconsistencia menor).

Evidencia en `src/analisis/genero.py` (línea 67):

```
1 base["pct_masculino"] = 1 - base["pct_femenino"]
```

2.0.9 `src/analisis/longitudinal.py`

Rol: analisis · **Líneas:** 207

Qué hace: Núcleo del análisis longitudinal (Issue #11): construye el panel investigador×convocatoria a partir de `ID_PERSONA_PR`, compara trayectorias entre periodos consecutivos (sube/baja/se mantiene/desaparece), calcula matrices de transición de conteos y probabilidades, y tasas de retención con entrada de nuevos investigadores. Cubre las 6 convocatorias históricas (2013-2021).

Valoración: mantener con mejoras — lógica sólida para el alcance descriptivo; la ampliación inferencial (markovianidad, supervivencia) es la recomendación de mayor valor.

Hallazgos relevantes:

- Ninguno crítico; riesgo metodológico: interpretar las transiciones como cadena de Markov sin validar la propiedad (caveat recomendado).

2.0.10 `src/analisis/produccion.py`

Rol: analisis · **Líneas:** 340

Qué hace: Análisis cruzado producción <-> investigadores (Sprint 5): cobertura de reconocimiento, productividad por categoría/género/área/territorio, mix de tipologías y reconocidos sin producción. Entrada: `prod` (3.166.629 filas, Socrata 33dq-ab5a) e `inv` (consolidado transformado), cruzados por `ID_PERSONA_PD` <-> `ID_PERSONA_PR` (+ `ID_CONVOCATORIA`); salida: DataFrames exportados a `evidencias/produccion_*.csv` y consumidos por `sprint5-produccion.py`.

Valoración: mantener/mejorar — módulo metodológicamente sólido y bien documentado; priorizar la robustez de tipos/años y la reproducibilidad del input crudo (o versionar un parquet de producción) para que el cruce sea reproducible sin 1,2 GB.

Hallazgos relevantes:

- `src/analisis/produccion.py:43`: `pd.to_datetime(ANO_CONVO)` sin manejo de ints/seriales -> año 1970 si el campo es numérico (inconsistente con `Transformacion.py:99-116`).

Evidencia en `src/analisis/produccion.py` (línea 43):

```
1 df["ANO_CONVO_INT"] = pd.to_datetime(df["ANO_CONVO"],
    errors="coerce").dt.year.astype("Int64")
```

Evidencia en `Transformacion.py` (líneas 99-116):

```
1 fecha = pd.to_datetime(s, dayfirst=True, errors="coerce")
2
3 # Intento 2: serial numérico de Excel
4 s_num = pd.to_numeric(s, errors="coerce")
5 fecha_serial = pd.to_datetime(
6     s_num, unit="D", origin="1899-12-30", errors="coerce"
7 [...])
```

- `src/analisis/produccion.py:199`: `pivot.rename` no falla si falta 'FEMENINO', pero `pivot['prom.femenino']` sí lanza `KeyError` en áreas sin mujeres.

Evidencia en `src/analisis/produccion.py` (línea 199):

```
1 pivot["razon_f_m"] = (pivot["prom_femenino"] / pivot["
    prom_masculino"]).round(3)
```

- `src/analisis/produccion.py:89`: división por `len(autores_unicos)` -> `ZeroDivisionError` si `prod` está vacío (caso límite).

Evidencia en `src/analisis/produccion.py` (línea 89):

```
1 "pct_autores_reconocidos": round(len(overlap) / len(
    autores_unicos) * 100, 2),
```

- `src/analisis/produccion.py:63-65`: `isin` usa IDs normalizados a `Int64`; si `id_persona_pr` llega con ceros a la izquierda desde el XLSX (leído como `str`) y `prod` como `int`, el cruce pierde matches (depende de la inferencia de tipos).

Evidencia en `src/analisis/produccion.py` (líneas 63-65):

```
1 ids_reconocidos = set(inv["ID_PERSONA_PR"].dropna().astype("
    Int64"))
2 p = prod.copy()
3 p["es_reconocido"] = p["ID_PERSONA_PD"].isin(ids_reconocidos)
```

2.0.11 `src/analisis/redes.py`

Rol: analisis · **Líneas:** 225

Qué hace: Análisis de redes de co-filiación institucional (Issues #15/#16): normaliza nombres de instituciones, extrae pares de instituciones de INST_FILIA (separador '|'), construye grafos NetworkX, calcula métricas (grado, betweenness, densidad, componentes) y genera HTML interactivo con Pyvis. Entrada: DataFrame transformado con INST_FILIA; salida: DataFrames de pares/nodos/aristas (evidencias/redes_*.csv) y HTML en hallazgos/.

Valoración: refactorizar — análisis con buen valor analítico, pero la pérdida de pares multi-filiación y el conteo duplicado de pesos comprometen las métricas publicadas (n_aristas, peso de aristas); corregir antes de citar resultados de red.

Hallazgos relevantes:

- src/analisis/redes.py:62-64: `str.split('|', n=1)` descarta afiliaciones 3+; red incompleta (subestimación de aristas y nodos).

Evidencia en src/analisis/redes.py (líneas 62-64):

```
1 multi[["inst_a", "inst_b"]] = (
2     multi["INST_FILIA"]
3     .str.split("|", n=1, expand=True)
```

- src/analisis/redes.py:74-79: `sorted()` sobre pares que contienen " (de 'A | ') crea un nodo vacío en el grafo; con tipos no comparables podría lanzar `TypeError`.

Evidencia en src/analisis/redes.py (líneas 74-79):

```
1 multi = multi[multi["inst_a"] != multi["inst_b"]].copy()
2 # Normalizar orden para que (A,B) == (B,A)
3 multi[["inst_a", "inst_b"]] = pd.DataFrame(
4     [sorted([a, b]) for a, b in zip(multi["inst_a"], multi["
5         inst_b"])],
6     index=multi.index,
```

- src/analisis/redes.py:93-98: el peso del grafo global cuenta filas investigador x convocatoria, no investigadores únicos: 'investigadores compartidos' queda inflado entre años.

Evidencia en src/analisis/redes.py (líneas 93-98):

```
1 for (a, b), grp in pares.groupby(["inst_a", "inst_b"]):
2     peso = len(grp)
3     if G.has_edge(a, b):
4         G[a][b]["weight"] += peso
5     else:
6         G.add_edge(a, b, weight=peso)
```

- src/analisis/redes.py:16-48: `normalizar_institucion` devuelve el valor original (posiblemente sin estandarizar) en el caso base.

Evidencia en src/analisis/redes.py (líneas 16-48):

```
1 def normalizar_institucion(nombre) -> str:
2     """
3     Colapsa variantes (sedes, parentesis, mayusculas) a una sola
4     entidad.
```

```

4
5     Reglas:
6     - Si hay parentesis y contiene una institucion (Universidad/
7       Instituto/etc),
8     [...]

```

2.0.12 src/analisis/territorial.py

Rol: analisis · **Líneas:** 107

Qué hace: Análisis de concentración territorial (Issue #13): índice Herfindahl-Hirschman (HHI) por convocatoria, top territorios y tabla de cuotas departamento x año. Entrada: DataFrame transformado; salida: DataFrames exportados a evidencias/territorial_*.csv.

Valoración: mantener — análisis correcto y exportable; mejorar la semántica del top-n y centralizar el filtro de faltantes.

Hallazgos relevantes:

- src/analisis/territorial.py:81-87: el top-n global combinado con pct anual puede producir tablas donde ningún año muestre el territorio #1 de ese año; no es error de cálculo, pero distorsiona el concepto de 'top territorios por convocatoria'.

Evidencia en src/analisis/territorial.py (líneas 81-87):

```

1     top = (
2         sub[col_geo]
3         .value_counts()
4         .head(n)
5         .index.tolist()
6     )
7     [...]

```

- src/analisis/territorial.py:44 y 71: si col_geo contuviera valores no-cadena (el NaN ya se dropea, pero p.ej. códigos numéricos), `.str.strip()` lanzaría `AttributeError` — depende del contrato de ingesta.

Evidencia en src/analisis/territorial.py (línea 44):

```

1     sub = sub[sub[col_geo].str.strip().str.upper() != "NO
        REPORTADO"]

```

2.0.13 scripts/sprint2_matrices_transicion.py

Rol: orquestacion · **Líneas:** 125

Qué hace: Calcula y visualiza las matrices de probabilidad de transición entre categorías (Junior/Asociado/Senior/Emérito) para los 5 periodos consecutivos de las 6 convocatorias

(2013-2021), en dos variantes: observada (solo investigadores retenidos) y extendida (incluye el estado 'Desaparece'). Usa `src/analisis/longitudinal` (`construir_panel`, `comparar_periodo`, `matriz_transicion`, `matrices_todos_periodos`). Salidas: 2 figuras de heatmaps en `artifacts/sprint2_transiciones/` y 20 CSVs (conteos y probabilidades $\times$ ext/obs $\times$ 5 periodos) en `evidencias/`.

Valoración: mantener — es la entrega central del análisis longitudinal y produce evidencia verificable en el clon; requiere eliminar el cómputo duplicado, advertir el sesgo de la matriz observada y limpiar la nomenclatura legacy.

Hallazgos relevantes:

- `scripts/sprint2_matrices_transicion.py:83-95` — las matrices se calculan dos veces (bucle de impresión y `matrices_todos_periodos`): doble costo de cómputo sobre 77.237 registros y riesgo de que las cifras impresas difieran de las exportadas si la función cambiara entre ambas llamadas.

Evidencia en `scripts/sprint2_matrices_transicion.py` (líneas 83-95):

```

1     for a0, a1 in zip(anios, anios[1:]):
2         comp = comparar_periodo(panel, a0, a1)
3         _, probs_obs = matriz_transicion(comp, incluir_desaparece=
4             False)
5         _, probs_ext = matriz_transicion(comp, incluir_desaparece=
6             True)
7         print(f"\n{'='*55}")
8         print(f"Periodo {a0}-{a1} -- Observada (solo retenidos):")
9     [...]
```

- `scripts/sprint2_matrices_transicion.py:49` — `plt.subplots(1, n)` con `n=1` devuelve un solo Axes (no iterable); el `zip(axes, periodos)` fallaría con `TypeError` si el panel tuviera un único periodo.

Evidencia en `scripts/sprint2_matrices_transicion.py` (línea 49):

```

1     fig, axes = plt.subplots(1, n, figsize=(5 * n, 4))
```

- `Evidencias/ legacy`: los archivos `matriz_transicion_extendida_2017_2019.csv`, `matriz_probabilidades_extendida_2019_2021.csv` y `matriz_transicion_2017_2019.csv` (sin prefijo) ya no son generados por este script; quedan como evidencia huérfana que puede confundir la trazabilidad.

2.0.14 `scripts/sprint2_panel_longitudinal.py`

Rol: orquestacion · **Líneas:** 229

Qué hace: Orquesta el análisis longitudinal del Sprint 2 (Issue #11): carga el consolidado (`cargar_consolidado` + `transformar`), construye el panel de investigadores por convocatoria con `src/analisis/longitudinal.py`, genera 7 figuras (evolución total, por género, por gran área, categorías, nuevos ingresos, tracking por periodo, tasas de retención) y exporta 4+ CSVs de evidencias.

Valoración: mantener — bien estructurado y verificable; solo requiere el consolidado completo de 6 convocatorias para reproducir los resultados del README.

Hallazgos relevantes:

- Ninguno crítico; dependencia implícita de que el consolidado tenga las 6 convocatorias para reproducir los 5 periodos del README (problema de datos, no de código).

2.0.15 scripts/sprint5_produccion.py

Rol: orquestacion · **Líneas:** 251

Qué hace: Análisis integral de producción de grupos: cobertura de reconocimiento (autores únicos y por convocatoria), productividad por categoría, brecha de género × gran área OCDE, concentración territorial, mix de tipologías y reconocidos sin producción. Usa src/ingesta (cargar_consolidado, cargar_produccion), src/Transformacion.transformar y 10 funciones de src/analisis/produccion. Entradas: investigadores consolidados + producción Socrata; salidas: 6 figuras PNG y 8 evidencias (7 CSVs + 1 JSON).

Valoración: mantener — es el script central del sprint 5, con salidas verificadas y documentadas; las mejoras prioritarias son unificar la definición de 'reconocido' entre scripts y validar la calidad del join antes de interpretar coberturas.

Hallazgos relevantes:

- src/analisis/produccion.py:63-65 (invocado desde sprint5_produccion.py:203) — cobertura_reconocidos marca es_reconocido sin condicionar la convocatoria: un producto de 2013 firmado por alguien reconocido solo en 2021 cuenta como 'firmado por reconocido', sobreestimando la cobertura frente a la definición por convocatoria del DuckDB (documentado en el docstring de la función como decisión, pero no advertido en el script orquestador).

Evidencia en src/analisis/produccion.py (líneas 63-65):

```

1     ids_reconocidos = set(inv["ID_PERSONA_PR"].dropna().astype("
2         Int64"))
3     p = prod.copy()
4     p["es_reconocido"] = p["ID_PERSONA_PD"].isin(ids_reconocidos)

```

Evidencia en sprint5_produccion.py (línea 203):

```

1     cov = cobertura_reconocidos(prod, inv)

```

- scripts/sprint5_produccion.py:75-78 — la etiqueta de porcentaje se posiciona en total+5000 y el bucle itera zip(n_reconocidos, n_productos, pct_reconocidos): si n_productos < 5000 la etiqueta queda fuera del rango visible del eje (cosmético).

Evidencia en scripts/sprint5_produccion.py (líneas 75-78):

```

1     for i, (rec, total, pct) in enumerate(

```

```

2         zip(cob_df["n_reconocidos"], cob_df["n_productos"], cob_df
3             ["pct_reconocidos"])
4     ):
        ax.text(i, total + 5000, f"{pct:.1f}%", ha="center",
                fontsize=9)

```

- scripts/sprint5_produccion.py:243-244 — import json dentro de main() (estilo menor; debería ir al tope con los demás imports).

Evidencia en scripts/sprint5_produccion.py (líneas 243-244):

```

1     with open(EVIDENCIAS / "produccion_resumen_cobertura.json", "w"
2               ", encoding="utf-8") as f:
        json.dump(cov_unicos, f, ensure_ascii=False, indent=2)

```

- src/analisis/produccion.py:38-40 — pd.to_numeric(errors='coerce') convierte IDs no numéricos a NA silenciosamente; si ID_PERSONA_PD tiene formatos mixtos, filas completas se pierden del análisis sin reporte.

Evidencia en src/analisis/produccion.py (líneas 38-40):

```

1     for c in ("ID_PERSONA_PD", "ID_CONVOCATORIA"):
2         if c in df.columns:
3             df[c] = pd.to_numeric(df[c], errors="coerce").astype("
                    Int64")

```

2.0.16 scripts/sprint6_tabla_maestra_ies.py

Rol: orquestacion · **Líneas:** 635

Qué hace: Construye el borrador de la tabla maestra canónica de IES a partir de evidencias/instituciones_unicas_raw.csv (dump de strings únicos de INST_FILIA con frecuencias), aplicando un diccionario MAPEO hardcodeado de ~250 claves string → (nombre_canonico, sigla, departamento_sede, naturaleza). Solo usa pandas, sin tocar el padrón ni producción. Salidas: tabla_maestra_ies.csv, mapping_inst_filia_to_ies.csv e ies_pendientes_revision.csv.

Valoración: refactorizar — como borrador cumple y sus productos existen, pero un catálogo canónico de IES no debería vivir en código: externalizar el mapeo a datos versionados, renombrar métricas y añadir cobertura mínima garantizada antes de que otros scripts (geografía, app) dependan de él en producción.

Hallazgos relevantes:

- scripts/sprint6_tabla_maestra_ies.py:607 — 'sedes_registradas' = nunique de departamento_sede: dos sedes en el mismo departamento (p. ej. dos campus de la UNAL en Bogotá) cuentan como 1, mientras dos sedes en departamentos distintos cuentan como 2; la métrica no es número de sedes.

Evidencia en scripts/sprint6_tabla_maestra_ies.py (línea 607):

```

1     sedes_registradas=("departamento_sede", "nunique"),

```

- scripts/sprint6_tabla_maestra_ies.py:38-562 — claves hardcodeadas sin validación de duplicados ni de normalización: si instituciones_unicas_raw.csv se regenera con otro case/acentos, la cobertura cae sin ninguna señal de error.

Evidencia en scripts/sprint6_tabla_maestra_ies.py (líneas 38-562):

```

1 MAPEO = {
2     # --- Top 20 por frecuencia
3     "UNIVERSIDAD DE ANTIOQUIA":
4         ("Universidad de Antioquia", "UdeA", "Antioquia", "publica"),
5     "UNIVERSIDAD NACIONAL DE COLOMBIA SEDE BOGOTA (UNIVERSIDAD NACIONAL DE COLOMBIA)":
6         ("Universidad Nacional de Colombia", "UNAL", "Bogotá D.C.", "publica"),
7     [...]

```

- scripts/sprint6_tabla_maestra_ies.py:592 — pd.concat([raw, asignaciones], axis=1) asume que aplicar_mapeo devuelve una Series por fila alineada por índice; correcto aquí, pero frágil si se reordenara raw antes.

Evidencia en scripts/sprint6_tabla_maestra_ies.py (línea 592):

```

1 asignaciones = raw.apply(aplicar_mapeo, axis=1)

```

- scripts/sprint6_tabla_maestra_ies.py:620-623 — ies_pendientes_revision.csv incluye todas las entradas sin mapeo, pero el script no imprime las 10 pendientes más frecuentes para facilitar la revisión manual (solo cobertura agregada).

Evidencia en scripts/sprint6_tabla_maestra_ies.py (líneas 620-623):

```

1 pendientes = mapping[mapping["estado"] == "pendiente_revision"]
2     .copy()
3     .to_csv(EVIDENCIAS / "ies_pendientes_revision.csv",
4             index=False, encoding="utf-8")
5 cobertura_pendientes = pendientes["n_apariciones"].sum() /
6     total_apariciones * 100

```

2.0.17 scripts/sprint6_validacion_calidad.py

Rol: orquestacion · **Líneas:** 121

Qué hace: Valida y documenta el tratamiento de valores atípicos de la variable EDAD_ANOS_PR del padrón: detecta los casos >100 (máximo observado 956, error de digitación), compara tres estrategias (original, eliminación, imputación por mediana de grupo) y genera evidencia auditable. Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y src/analisis/calidad (detectar_atipicos_edad, resumen_tratamiento, filtrar, imputar_mediana, UMBRAL_EDAD_MAX). Salidas: 2 CSVs, 1 JSON y 1 figura.

Valoración: mantener — cumple su propósito (decisión metodológica auditable) con evidencia verificada; requiere solo ajustes menores de consistencia de conteos y redondeo.

Hallazgos relevantes:

- scripts/sprint6_validacion_calidad.py:62-84 — edad_stats mezcla unidades: len(df) cuenta filas (incluye NaN en EDAD) mientras mean/median/std de pandas ignoran NaN; si hubiera filas con edad faltante, n_registros y las demás columnas no serían comparables.

Evidencia en scripts/sprint6_validacion_calidad.py (líneas 62-84):

```

1  edad_stats = pd.DataFrame({
2      "metodo": ["original", "eliminacion (>100)", "imputacion
3      mediana por grupo"],
4      "n_registros": [len(df), len(df_filtrado), len(df_imputado)
5      ]),
6      "media": [
7          round(df["EDAD_ANOS_PR"].mean(), 2),
          round(df_filtrado["EDAD_ANOS_PR"].mean(), 2),
          [...]

```

- scripts/sprint6_validacion_calidad.py:70-73 — df['EDAD_ANOS_PR'].median() sin round frente a round(..., 2) en media (formato inconsistente; en el CSV aparece 43.29 por coincidencia del valor).

Evidencia en scripts/sprint6_validacion_calidad.py (líneas 70-73):

```

1      "mediana": [
2          df["EDAD_ANOS_PR"].median(),
3          df_filtrado["EDAD_ANOS_PR"].median(),
4          df_imputado["EDAD_ANOS_PR"].median(),

```

- scripts/sprint6_validacion_calidad.py:50-51 — detectar_atipicos_edad solo imprime, no informa el conteo de NaN de EDAD_ANOS_PR, que podría alterar la interpretación del umbral.

Evidencia en scripts/sprint6_validacion_calidad.py (líneas 50-51):

```

1  atipicos = detectar_atipicos_edad(df)
2  print(atipicos.to_string(index=False))

```

2.0.18 streamlit_app.py

Rol: visualizacion · **Líneas:** 983

Qué hace: Dashboard Streamlit principal del Observatorio MinCiencias con 7 pestañas tipo capítulo (Calidad, Trayectoria, Territorio, Diversidad, Redes, Productividad, Datos crudos). Lee el consolidado vía src/ingesta.cargar.consolidado (XLSX de respaldo en datos/tarea-join), lo transforma con src/Transformacion.transformar y lo cruza con ~30 funciones de src/analisis/*. Cada sección sigue el patrón pregunta ->evidencia ->hallazgo ->caveat, con filtros globales en la barra lateral.

Valoración: mejorar: es la pieza central del proyecto, bien estructurada y con 5 de 7 secciones funcionales con el XLSX actual. Corregir el bug del HHI, robustecer la cache,

documentar/proveer el dataset de produccion y eliminar los hardcodes antes de considerar un refactor.

Hallazgos relevantes:

- Sección 3: `hhi_por_convocatoria(df, col_geo=col).reset_index()` devuelve 5 columnas (`index` + `ANO_CONVOCO_INT` + `hhi` + `n_total` + `n_territorios`) y luego `hhi_df.columns = ['Año', 'HHI']` lanza `ValueError 'Length mismatch'`; el `except` lo captura y el gráfico HHI nunca se muestra (líneas 434-435)
- Sección 6: depende de `datos/raw/produccion_grupos.csv` que no está en el repo; toda la sección (4 métricas + 6 gráficos) se degrada a un warning
- Sección 1: fallback hardcodeado `'else 16796'` y umbral mágico `ID_CONVOCATORIA == 20` sin validación -> número potencialmente incorrecto en silencio
- `panel_completo(len(df), df)` usa `len(df)` como hash de cache -> colisiones entre filtros del mismo tamaño (línea 347)
- Si el CSV de producción existiera pero vacío, `cov.iloc[0]/cov.iloc[-1]` (líneas 761-762) lanzarían `IndexError` no capturado y crashearía la app
- `Caption` y `docstring` desincronizados: '6 secciones' en el header (línea 7) vs 7 tabs reales

3 Relación con los demás criterios

Este documento se centra en el criterio 3. Los demás criterios de la rúbrica — Entendimiento del objetivo del repositorio; Datos utilizados: suficiencia y corrección; Validación de la calidad estadística; Lógica de creación del repositorio y Línea de tiempo de commits — se desarrollan a fondo en sus propios documentos y en el **documento maestro** de la auditoría (`maestro_auditoria.pdf`), que los integra todos con el detalle completo de datos, código, estadística, lógica y línea de tiempo. Esta separación evita repetir contenido: cada PDF profundiza en lo suyo.

4 Conclusiones y veredicto del criterio (versión LITE)

Hallazgo

Veredicto: El código está bien construido en general (modular, documentado y trazable) y los 51 archivos quedan documentados uno por uno; 10 de los 16 scripts principales corren en un clon fresco y los 6 fallos tienen causa identificada (datos no versionados y una dependencia no declarada).

4.1 Hallazgos de mayor impacto

- Bug del HHI en el dashboard (`ValueError` tragado) y denominador de minorías en `comparar_dane()`: el comentario no coincide con el código.

- Reproducibilidad: el flujo ETL de CI/CD referencia módulos inexistentes, el Dockerfile despliega un placeholder y kaleido no está declarado.

Esta versión resume el criterio; el análisis completo, las fichas por archivo y la evidencia están en el documento completo del criterio.

Referencias
